TIẾT 2

CHUỖI KÝ TỰ, KÝ TỰ THOÁT (ESCAPE SEQUENCES)

& KỸ NĂNG CHẨN ĐOÁN, GỠ LỖI LẬP TRÌNH (DEBUGGING)

Thời lượng: 50 phút | 35’ Lý thuyết & Thảo luận + 15’ Hands-on Lab 2

Chuỗi ký tự (String Literals) & Thách thức khi xuất ký tự đặc biệt

Chuỗi ký tự (String Literal)

- Là một chuỗi các ký tự nằm trong cặp **dấu ngoặc kép** (`"..."`).
- Ví dụ: `"Xin chao"`, `"IT26M"`, `""` (chuỗi rỗng).
- **Cấm:** Không được ngắt một chuỗi qua nhiều dòng mà không đóng ngoặc kép!

Vấn đề hóc búa nảy sinh:

Nếu muốn in ra màn hình chính xác câu sau:

```
Giao vien noi: "Java rat thu vi!"
```

Nếu viết:

```
System.out.println("Giao vien noi: "Java rat thu vi!"");
```

→ **Trình biên dịch sẽ báo lỗi ngay!** Vì nó nghĩ chuỗi kết thúc ở dấu ngoặc kép thứ hai.

Giải pháp trong mọi ngôn ngữ lập trình: Sử dụng **Ký tự thoát (Escape Sequences)**!

Bảng ký tự thoát chuẩn trong Java (Escape Sequences)

Quy tắc: Dấu gạch chéo ngược (\) kết hợp với ký tự đi liền sau:

Ký tự thoát	Tên gọi	Ý nghĩa & Hành vi thực hiện
`\t`	Tab	Thêm khoảng thụt lề tab (căn chỉnh cột dữ liệu đẹp mắt).
`\n`	Newline	Xuống dòng mới ngay tại vị trí xuất hiện.
`\"`	Double quote	In dấu ngoặc kép " mà không làm đóng chuỗi.
`\\`	Backslash	In một dấu gạch chéo ngược \.

Listing 3: Ví dụ sử dụng Escape Sequences

```
1 System.out.println("Cot 1\tCot 2\tCot 3");
2 System.out.println("Dong 1\nDong 2\nDong 3");
3 System.out.println("Dau \"nhay kep\" va gach \\");
4
```

Kết quả in ra màn hình:

```
Cot 1   Cot 2   Cot 3
Dong 1
Dong 2
Dong 3
Dau "nhay kep" va gach \
```

Định danh (Identifiers) & Từ khóa chuẩn Java

Định danh (Identifiers)

Tên do lập trình viên đặt cho lớp, phương thức, biến...

- **Ký tự hợp lệ:** Các chữ cái (`a-z, A-Z`), chữ số (`0-9`), dấu gạch dưới (_), dấu đô la ($).
- **Quy tắc 1: Không được bắt đầu bằng chữ số!** (Hợp lệ: `diem1`, không hợp lệ: `1diem`).
- **Quy tắc 2:** Không chứa dấu cách hoặc ký tự đặc biệt (`@, #, %, -, +`).
- **Quy tắc 3: Không trùng với Từ khóa Java!**

Từ khóa Java (Keywords – Dành riêng)

Java có 50+ từ khóa đặc biệt đã được định nghĩa sẵn cú pháp:

`public`	`class`	`static`	`void`
`int`	`double`	`boolean`	`char`
`if`	`else`	`for`	`while`
`return`	`new`	`true`	`false`

Tuyệt đối không dùng các từ trên làm tên Class hay tên hàm!

Phong cách lập trình chuẩn đại học: Quy ước CamelCase & Comments

Quy ước đặt tên CamelCase

- **Tên Lớp (UpperCamelCase):**
  Viết hoa chữ cái đầu của mỗi từ.
  Ví dụ: `Hello`, `DrawFigures3`, `StudentManager`.
- **Tên Phương thức & Biến (lowerCamelCase):**
  Chữ cái đầu viết thường, viết hoa các từ tiếp theo.
  Ví dụ: `main`, `drawDiamond`, `calculateTotalScore`.

3 Loại chú thích (Comments) trong Java

- **Chú thích 1 dòng:** Bắt đầu bằng `//`
  `// Tinh tong diem cua sinh vien`
- **Chú thích nhiều dòng:** Cặp `/* ... */`
  ```
  /* Tac gia: IT26M
     Ngay tao: 2026 */
  ```
- **Chú thích tài liệu (Javadoc):** Cặp `/** ... */`
  Dùng để sinh tài liệu API tự động.

*"Mã nguồn viết ra cho người đọc hiểu trước, máy tính hiểu sau." – Thụt lề (**Indentation**) nhất quán 4 khoảng trắng!*

1.3 Phân loại lỗi lập trình: 3 nhóm lỗi kinh điển

Mọi lập trình viên từ sinh viên năm nhất đến chuyên gia thâm niên đều phải đối mặt với lỗi!

1. Lỗi Cú pháp (Syntax Errors)

- Xảy ra trong giai đoạn **Biên dịch (Compile-time)**.
- Vi phạm ngữ pháp ngôn ngữ: thiếu dấu `;`, sai chính tả từ khóa, quên đóng ngoặc `}`.
- **Đặc điểm:** Trình biên dịch `javac` phát hiện và từ chối sinh file `.class`.

2. Lỗi Thực thi (Runtime Errors)

- Biên dịch thành công, xảy ra khi **Đang chạy (Run-time)**.
- Chương trình yêu cầu CPU thực hiện phép toán bất khả thi (chia cho 0, truy cập mảng rỗng).
- **Đặc điểm:** Chương trình bị sập (*crash*) và tung ra ngoại lệ (**Exception**).

3. Lỗi Ngữ nghĩa (Logic / Intent Errors)

- Chương trình biên dịch tốt, chạy không bị sập.
- **Nhưng kết quả sai** so với yêu cầu ban đầu!
- Ví dụ: Cần tính tiền thuế mà viết nhầm dấu `+` thành dấu `-`.
- **Đặc điểm:** Nguy hiểm nhất vì máy tính không thể tự phát hiện!

Kỹ năng đọc thông báo lỗi từ Trình biên dịch (`javac`)

Cấu trúc một thông báo lỗi điển hình:

```
Hello.java:3: error: ';' expected
        System.out.println("Hello, world!")
                                           ^
1 error
```

Các thành phần của thông báo lỗi

1. **Tên file bị lỗi:** `Hello.java`
2. **Dòng bị lỗi:** Dòng số `3`
3. **Loại lỗi:** `';' expected` (Thiếu dấu chấm phẩy)
4. **Vị trí con trỏ (^):** Chỉ ra điểm chính xác mà compiler phát hiện bất thường.

Kinh nghiệm vàng khi Debug:

- **Quy tắc 1:** Luôn nhìn vào **lỗi đầu tiên** trong danh sách lỗi! Một lỗi ở dòng 3 có thể gây ra hiệu ứng domino tạo 10 lỗi giả phía sau.
- **Quy tắc 2:** Nếu dòng báo lỗi trông hoàn toàn đúng, hãy kiểm tra **dòng ngay phía trước nó**.

[CHECKPOINT 2] Thử tài thám tử chẩn đoán lỗi

Đoạn chương trình sau chứa 3 lỗi cú pháp. Bạn hãy tìm ra chúng?

```
1 public MyProgram {
2     public static void Main(String[] args) {
3         System.out.println("Lop IT26M)
4     }
5 }
6
```

Chẩn đoán và phân tích chi tiết:

1. **Lỗi dòng 1:** Thiếu từ khóa **`class`** giữa `public` và `MyProgram` → `public class MyProgram`.
2. **Lỗi dòng 2:** Tên hàm `main` bị viết hoa chữ M → Java phân biệt hoa thường, phải là **`main`**.
3. **Lỗi dòng 3:** Chuỗi ký tự thiếu dấu ngoặc kép đóng `"` và dấu chấm phẩy `;` kết thúc lệnh → `System.out.println("Lop IT26M");`.

[HANDS-ON LAB 2] Thực hành Debug & Chuỗi ký tự thoát

Thử thách thực hành Tiết 2 (Thời gian: 15 phút – Trực tiếp trên máy tính)

Bài toán: Viết chương trình Java hoàn chỉnh tên là `EscapeChallenge.java` để in ra màn hình chính xác đoạn văn bản sau (chú ý từng dấu ngoặc kép, gạch chéo và tab):

Kết quả mong đợi trên Console:

```
Dac ta he thong:
    1. Thu muc: C:\Java\IT26M
    2. Khau hieu: "Hoc la phai hanh!"
    3. Tam giac: /\  /\
```

Gợi ý kỹ thuật:

- Dấu gạch chéo ngược trong đường dẫn: Dùng `\\`.
- Dấu ngoặc kép trong khẩu hiệu: Dùng `\"`.
- Thụt lề đầu dòng: Dùng `\t`.
- Hãy thử cố tình gõ thiếu dấu `\` để xem trình biên dịch báo lỗi gì!

TIẾT 3

PHÂN RÃ THỦ TỤC (PROCEDURAL DECOMPOSITION)

PHƯƠNG THỨC TĨNH & TIẾN HÓA CASE STUDY DRAWFIGURES

Thời lượng: 50 phút | 30’ Lý thuyết thiết kế + 20’ Mini Project Lab 3

1.4 Vấn đề của bài toán lớn & Tư duy Phân rã Top-down

Cạm bẫy của chương trình nguyên khối (Monolithic)

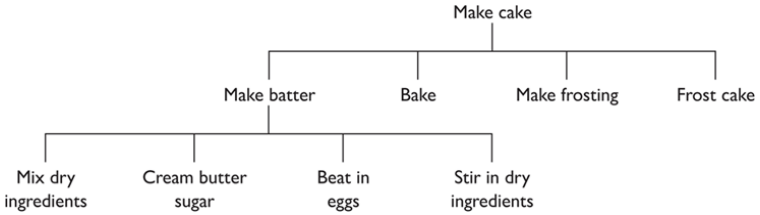
- Đặt tất cả hàng trăm dòng lệnh vào phương thức `main`.
- Mã nguồn cực kỳ khó đọc, khó hiểu (*spaghetti code*).
- **Trùng lặp mã nguồn (Code Duplication):** Cùng một khối lệnh phải gõ đi gõ lại nhiều lần!
- Rất khó sửa lỗi và không thể tái sử dụng.

Nguyên lý Phân rã Top-down (Divide and Conquer)

- Chia bài toán lớn phức tạp thành các **bài toán con (subtasks)**.
- Tiếp tục chia nhỏ bài toán con cho đến khi đạt được các thao tác đơn giản nhất (*primitives*).
- Giải quyết từng bài toán con bằng các **phương thức độc lập**.

Nguyên lý cốt lõi trong kỹ nghệ phần mềm: DRY – Don't Repeat Yourself!

Ví dụ thực tế: Phân rã tác vụ “Làm bánh” (Make Cake)



Hình 1.2: Sơ đồ cấu trúc phân rã tác vụ làm bánh (Trích giáo trình Chapter 1, Trang 66)

Đặc điểm của Sơ đồ Cấu trúc (Structure Diagram)

- **Cấp cao nhất (Top level):** Nhiệm vụ tổng thể (*Make cake*).
- **Cấp 1:** Các giai đoạn chính (*Make batter*, *Bake*, *Make frosting*, *Frost cake*).
- **Cấp 2:** Thao tác cụ thể (*Mix dry ingredients*, *Cream butter and sugar*...).

Ý nghĩa sư phạm trong lập trình

- Giúp người lập trình có cái nhìn tổng quan trước khi viết từng dòng code.
- Trong Java, mỗi hộp trong sơ đồ cấu trúc sẽ tương ứng với một **phương thức tĩnh (static method)**.

Phương thức tĩnh (Static Methods): Cú pháp Khai báo & Lời gọi

Cú pháp Khai báo (Declaring a Method)

Định nghĩa tên và hành vi của phương thức:

```
public static void <tenPhuongThuc>() {
    <cac cau lenh thuc thi>;
}

```

Ví dụ:

```
public static void drawBox() {
    System.out.println("+------+");
    System.out.println("|      |");
    System.out.println("|      |");
    System.out.println("+------+");
}

```

Cú pháp Lời gọi (Calling a Method)

Ra lệnh cho máy tính thực thi phương thức:

```
<tenPhuongThuc>();

```

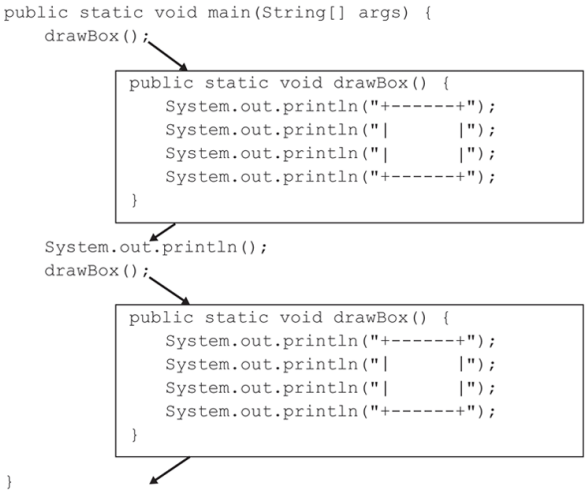
Ví dụ trong `main`:

```
public static void main(String[] args) {
    drawBox(); // Goi lan 1
    System.out.println();
    drawBox(); // Goi lan 2 (Tai su dung!)
}

```

Lưu ý

Luồng điều khiển khi gọi phương thức (Flow of Control)

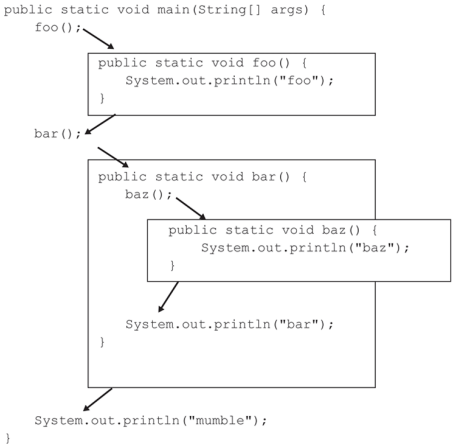


Sơ đồ luồng điều khiển chương trình DrawBoxes (Trích giáo trình Chapter 1, Trang 76)

Quy tắc chuyển giao điều khiển:

1. Chương trình **luôn bắt đầu** tại dòng đầu tiên của hàm `main`.
2. Khi gặp lời gọi hàm (`drawBox();`):
   - Máy tính **tạm dừng** thực thi tại `main`.
   - Nhảy tới thân phương thức `drawBox` và thực thi tuần tự các lệnh trong đó.
3. Khi phương thức chạy xong dấu ngoặc đóng `}`:
   - Quyền điều khiển quay trở lại (**return**) đúng vị trí vừa rời đi ở `main`.
   - Tiếp tục thực thi câu lệnh kế tiếp!

Luồng điều khiển đa cấp & Khái niệm Ngăn xếp gọi hàm (Call Stack)



Sự mở rộng luồng điều khiển lồng nhau foo → bar → baz (Trang 82)

Cơ chế hoạt động của Call Stack

- Một phương thức có thể gọi một phương thức khác (`bar()` gọi `baz()`).
- Máy tính ghi nhớ thứ tự quay về bằng cấu trúc **Ngăn xếp (Call Stack - LIFO: Vào sau ra trước)**.
- Khi `baz()` chạy xong → quay về `bar()` → `bar()` chạy xong → quay về `main`.

Lợi ích kép của Phân rã thủ tục:

1. **Structure (Cấu trúc):** Biến chương trình dài phức tạp thành các module ngắn gọn, dễ hiểu.
2. **Eliminate Redundancy (Xóa bỏ trùng lặp):** Viết 1 lần, gọi thực thi vô số lần!